

VEKTORLABS

Technical System Design Reference

Microstructure · Regime Detection · Alternative Data · LLM Reasoning

Field	Value
Version	1.0
Audience	Solo founder — internal build reference
Domain	vektorlabs.xyz
Stack	Python 3.11 · FastAPI · TimescaleDB · Redis · React
Pricing	\$47/mo (standard) · \$97/mo (unlimited symbols)

Table of Contents

1. System Overview
2. Architecture
3. ETL Pipeline — ingestion/
4. Module 1 — Microstructure Modeler
5. Module 2 — Regime Detector
6. Module 3 — Alternative Data Intelligence
7. Module 4 — LLM Financial Reasoner
8. Output Layer — API + Dashboard
9. Per-User Dashboard + Dynamic Symbols
10. Storage Design
11. Configuration — No YAML
12. Infrastructure + Deployment
13. Data Sources Reference
14. Build Order + Study Map

1. System Overview

VektorLabs is a real-time market intelligence terminal for semi-professional crypto futures traders. It solves the single most expensive mistake retail and semi-pro traders make: applying the right strategy in the wrong market condition.

The system continuously classifies what kind of market is currently happening, measures institutional order flow toxicity, aggregates alternative data signals, and extracts causal insight from text. Every output is regime-aware — signals are only surfaced when they are valid in the current market context.

Core premise: one trader avoiding one bad \$5,000 trade covers 50+ months of subscription. The pitch is ROI, not features.

Target Customer

Semi-professional crypto futures traders managing \$50k–\$500k. They understand microstructure terminology, have been burned by black-box signal services, and will pay for something explainable and verifiable. Reachable on Twitter/X, Discord, and prop firm communities.

What the System Produces

Output	Description	Frequency
Regime State	Current regime + confidence + transition probability	Per tick
OFI / VPIN	Order flow toxicity and imbalance	Per tick
Spread Decomposition	Adverse selection vs inventory cost	Per tick
Alt Data Confluence	Cross-signal score: on-chain + sentiment + macro	1–60 min
LLM Causal Insight	Extracted causal chains from news and filings	On event
Discord Alerts	Threshold-triggered notifications per user config	On event

2. Architecture

The system is divided into four horizontal layers. Each layer has one responsibility and communicates with the next through defined interfaces. No layer bypasses another. The hot path — from WebSocket tick to Redis publish — never touches PostgreSQL.

```
EXTERNAL SOURCES Binance WS | Reddit | FRED | Blockchain.com | NewsAPI | SEC EDGAR | v INGESTION  
LAYER (ingestion/) extract.py --> transform.py --> load.py | | v v TimescaleDB (historical) Redis  
Streams (real-time bus) | ████████████████████████████████████████████████████████████████ v v v  
Microstructure Regime Detector AltData Intel Modeler LLM Reasoner  
██████████████████████████████████████████████████████████████████████████████████ | v OUTPUT LAYER FastAPI REST |  
WebSocket | React | Alerts
```

Key Architectural Decisions

- **Redis Streams as signal bus.** Every module publishes to Redis Streams. No module calls another directly. Full decoupling — a module crash does not affect other modules.
- **TimescaleDB for history only.** Modules query TimescaleDB for offline training and backtesting. Real-time inference reads exclusively from Redis.
- **Hot path is PostgreSQL-free.** The path from WebSocket tick to Redis publish has zero database writes. DB writes are async background tasks.
- **No YAML configuration.** All behavioural config lives in the database as key-value pairs. Loaded to memory at startup. Refreshable without restart.
- **Dynamic symbol subscriptions.** Users add any valid Binance futures symbol. WebSocket streams spin up and shut down dynamically based on active subscribers.

3. ETL Pipeline — ingestion/

The ingestion layer is the data transport backbone of the entire system. It runs continuously, 24/7, and is the only part of the system that talks directly to external data sources. Three files. Clean separation.

extract.py — Pull raw data

Manages all external connections. Opens WebSocket streams, fires HTTP polling tasks, handles reconnection logic. Does not know what the data means — it only moves bytes reliably from source to the next stage.

Responsibilities:

- Maintain Binance L2 order book WebSocket stream per active symbol.
- Maintain Binance aggregated trade stream per active symbol.
- Poll Binance Futures REST endpoints for funding rate, open interest, and liquidation data on a configurable interval.
- Delegate to altdata ingestors (reddit.py, fred.py, google_trends.py, on_chain.py) for their respective sources.
- Reconnect with exponential backoff on any connection failure.
- Validate all incoming data at the boundary — reject malformed messages before they reach transform.

transform.py — Normalise and type

Converts raw bytes and dicts into typed Python dataclasses. Applies z-score normalisation to alternative data signals against a rolling baseline window. Stateless — no I/O, no side effects.

Output dataclasses:

Dataclass	Contents
OrderBookSnapshot	Symbol, timestamp, bids/asks, bid-ask imbalance, mid price
TradeEvent	Symbol, timestamp, price, qty, buyer-maker flag
FuturesSnapshot	Symbol, funding rate, open interest, liquidation data
AltDataPoint	Source, value, z-score vs rolling baseline, raw payload

load.py — Write to storage

Receives typed dataclasses from transform and writes them to two destinations: TimescaleDB for persistence and Redis Streams for real-time consumption. Every Redis Stream has an explicit maxlen to prevent unbounded memory growth.

Destination	What is written	maxlen / retention
TimescaleDB	All dataclasses — full historical record	30 days raw ticks
Redis Streams	Microstructure + AltData signals	10,000 / 1,000 msgs
Redis Cache	Latest state per symbol (TTL 60s)	60 second TTL

ingestion/ingestors/ — Pure transport clients

Three files that contain zero domain logic. They know how to open a connection and receive data. Nothing more.

- **binance_ws.py** — WebSocket stream manager with dynamic symbol subscription. Maintains a dict of active streams keyed by symbol.
- **binance_futures.py** — REST polling client for funding rates, open interest, and liquidation endpoints.
- **on_chain.py** — REST client for Blockchain.com stats API and Mempool.space fee and UTXO endpoints.

4. Module 1 — Microstructure Modeler

The microstructure module models price formation mechanics at the order level. It consumes raw order book and trade data from Redis Streams and produces quantitative metrics that describe the behaviour of institutional participants in the market right now.

This module provides the foundational features that feed directly into the Regime Detector. Without accurate microstructure features, regime classification has nothing meaningful to classify.

order_book.py — L2 book maintenance

Maintains a live L2 order book using a sorted data structure. Processes incremental depth updates from Binance WebSocket. Handles sequence number validation and book resync on gap detection.

- Bid and ask sides maintained as sorted price-level structures.
- Top-N levels configurable per symbol (BTC: 50, altcoins: 20).
- Detects and handles sequence gaps — triggers full book resync automatically.

features.py — Core microstructure metrics

Metric	What it measures	Why it matters
OFI	Net buy minus sell pressure weighted by depth change	Predicts short-term price direction
VPIN	Probability of trading against informed participant	Detects toxic flow before price moves
Kyle Lambda	Price impact per unit of signed order flow	Measures market depth and resilience
Trade Intensity	Order arrival rate vs cancellation rate	Identifies urgency in institutional flow
CVD	Cumulative volume delta — running buy/sell imbalance	Trend confirmation in order flow
VWAP Dev	Deviation of current price from volume-weighted avg	Institutional reference price distance

spread.py — Glosten-Milgrom decomposition

Decomposes the bid-ask spread into two components using the Glosten-Milgrom framework. This tells you whether the spread is wide because dealers fear informed traders (adverse selection) or because they are managing inventory risk.

- **Adverse selection component** — cost of trading against a participant who knows more than the dealer. High values indicate informed flow is present.
- **Inventory cost component** — cost of the dealer holding undesired inventory. High values indicate one-sided pressure from uninformed flow.

trade_flow.py — Trade stream analysis

- Classifies each trade as buyer-initiated or seller-initiated using the Lee-Ready algorithm applied to the aggregated trade stream.
- Computes rolling buy ratio — percentage of volume that is buyer-initiated over a configurable window.
- Detects absorption events — large passive orders absorbing aggressive flow without price movement. Classic institutional accumulation signal.

5. Module 2 — Regime Detector

The regime detector is the brain of the system. It determines what kind of market is currently operating so that every other module can contextualise its output accordingly. A signal that is valid in a trending regime is meaningless or actively dangerous in a mean-reverting regime.

A rule-based FSM tells you when a rule fires. An HMM tells you the probability that you are in a state. That probability is the edge.

Why HMM, not FSM

A Finite State Machine transitions deterministically when a threshold is crossed. It has no uncertainty. An HMM models the market regime as a hidden (unobservable) state that generates observable features with some probability distribution. This means:

- The system outputs a probability distribution over regimes, not a binary flag.
- Regime transitions are probabilistic — the model knows when it is uncertain.
- The model learns transition patterns from historical data via Baum-Welch rather than requiring hand-tuned thresholds.
- Regime confidence scores give traders a measure of how reliable the current classification is before acting on it.

The Four Regimes

Regime	Observable Characteristics	Trading Implication
Trending	High OFI magnitude, directional CVD, low mean-reversion	Momentum strategies valid
Mean-Reverting	Low OFI, oscillating CVD, high VWAP mean-reversion	Counter-trend entries valid
Volatile	High VPIN, wide spreads, large adverse selection component	Reduce size, widen stops
Illiquid	Thin book, high Kyle Lambda, erratic trade intensity	Avoid entry entirely

features.py — Regime feature matrix

Builds the observation vector fed into the HMM at each time step. Features are drawn from microstructure output plus price-derived measures.

- Realised volatility — rolling standard deviation of log returns.
- Parkinson volatility estimator — uses high-low range for efficiency.
- Trend strength — ADX-equivalent computed from order flow rather than price.
- Liquidity score — composite of book depth, spread width, and Kyle Lambda.
- OFI magnitude — absolute order flow imbalance normalised by volume.

hmm.py — Classifier

Wraps hmmlearn's GaussianHMM with 4 hidden states and full covariance. Trained offline on historical feature matrices via Baum-Welch EM. At runtime, uses the Viterbi algorithm to decode the most probable state sequence and outputs posterior probabilities for each regime.

trainer.py — Offline training

Pulls historical feature matrices from TimescaleDB and trains the HMM. Uses walk-forward validation to prevent overfitting. Retraining is triggered on a configurable schedule (default 24 hours) via the scripts/train_hmm.py script. Model files are stored with HMAC signatures to prevent tampering.

predictor.py — Real-time inference

Consumes the latest feature vector from Redis, runs forward algorithm to compute posterior regime probabilities, and publishes the result to the regime Redis Stream. Outputs regime label, confidence score, time-in-regime counter, and full transition probability matrix.

6. Module 3 — Alternative Data Intelligence

The alternative data module aggregates signals from sources that do not look like financial data. Its purpose is to detect when external forces are about to influence price before that influence appears in the order book.

The critical design principle here is velocity over absolute value. A Reddit sentiment score of 70 means nothing. A Reddit sentiment score that has increased 3 standard deviations in 2 hours means the narrative is shifting.

modules/altdata/ingestors/ — Domain-aware fetchers

File	Source	What it fetches	Interval
reddit.py	Reddit (PRAW)	Hot posts: score, comments, created time	5 min
fred.py	FRED API	CPI, yield curve, unemployment, T10Y2Y	24 hrs
google_trends.py	pytrends	Search volume for user-defined keywords	1 hr
on_chain.py	Blockchain.com Mempool.space	Exchange volume, fees, UTXO set	15 min

extractors/ — Signal computation

Three extractors process raw ingestor output into actionable signals. All operate on z-scores against rolling baselines, not absolute values.

sentiment.py

- Computes sentiment velocity: rate of change of sentiment score, not the score itself.
- Tracks post volume separately from score — volume spikes without sentiment shift indicate noise. Both moving together indicates real narrative formation.
- Window configurable per source (Reddit default 30-period rolling baseline).

spike_detector.py

- Applies MAD (Median Absolute Deviation) z-scoring — robust to fat tails which are common in alternative data series.
- Flags spikes above a configurable z-score threshold (default 2.5 sigma).
- Distinguishes between isolated spikes (noise) and sustained elevation (signal) using a streak counter.

macro_overlay.py

- Classifies current macro environment as risk-on or risk-off using FRED data.
- Primary indicators: T10Y2Y (yield curve), CPI trend, unemployment rate trend.
- Output: binary risk flag + composite macro score. Feeds into regime features.

confluence.py — Cross-signal correlation engine

The confluence engine is what differentiates this system from a simple aggregator. It computes cross-signal correlations and determines which alternative data signals historically lead price movement versus lag it.

- Granger causality tests run offline to classify each signal as leading or lagging.
- Confluence score computed when multiple leading signals align in the same direction.
- Score is regime-weighted — confluence in a trending regime scores higher than the same confluence in an illiquid regime.
- Published to Redis Stream for consumption by the dashboard and alert system.

7. Module 4 — LLM Financial Reasoner

The LLM reasoner extracts structured causal insight from unstructured text. It does not perform sentiment analysis — that is a solved, commoditised problem. It extracts causal chains: what caused what, therefore what is likely next.

The difference between sentiment analysis and causal extraction: sentiment says the CEO sounded nervous. Causal extraction says the CEO signalled margin compression, which implies reduced buyback capacity, which historically precedes 3-6 month underperformance.

fetchers/ — Text source clients

File	Source	What it fetches
sec_edgar.py	SEC EDGAR	8-K and 10-Q filings for tracked entities
news.py	NewsAPI	Headlines and article bodies for tracked assets
transcript.py	EDGAR	Earnings call transcripts (8-K Item 2.02)

prompts/ — Reasoning templates

Each prompt file contains a carefully engineered system prompt that instructs the LLM to reason in a specific way. Prompts are structured to force causal chains, not summaries.

causal_extractor.py

- Instructs the model to identify: triggering event, causal mechanism, affected entity, expected direction, and confidence level.
- Forces structured JSON output — no prose responses.
- Includes negative examples in the prompt to prevent summarisation.

fed_parser.py

- Specialised prompt for Federal Reserve statements and FOMC minutes.
- Extracts: hawkish/dovish shift, rate path expectations, balance sheet signals.
- Outputs confidence score based on language certainty markers.

contradiction.py

- Compares current filing language against previous filing language for same entity.
- Flags significant shifts in forward guidance, risk factor language, or revenue characterisation.
- Contradiction score: 0 (no change) to 1 (material contradiction).

reasoner.py — Main engine

Orchestrates the fetchers and prompts. Maintains a queue of text sources to process. Rate-limits LLM calls to stay within OpenRouter free tier. Publishes CausalInsight objects to Redis Stream on completion.

- Model: mistralai/mistral-7b-instruct:free via OpenRouter (default).
- Fallback: meta-llama/llama-3.1-8b-instruct:free.
- Output dataclass: CausalInsight (summary, affected assets, direction, confidence, time horizon, source).

8. Output Layer — API + Dashboard

The output layer consumes from all four Redis Streams and surfaces the information to users via a REST API, a WebSocket push channel, and a React dashboard. The alert system runs as a separate async task.

api/ — FastAPI server

Endpoint	Method	Description
/api/regime/current	GET	Current regime state, confidence, transition matrix
/api/microstructure/current	GET	Latest OFI, VPIN, spread decomposition
/api/altdata/confluence	GET	Current confluence score + signal breakdown
/api/insights/latest	GET	Latest LLM causal insights
/api/backtest/run	POST	Run regime-adjusted backtest on historical data
/api/users/{username}/settings	GET	Fetch user settings from database
/api/users/{username}/settings	PUT	Update user settings
/api/users/{username}/symbols	POST	Add symbol to user watchlist (validated)
/api/users/{username}/symbols	DELETE	Remove symbol, decrement stream subscriber count
/api/auth/login	POST	JWT authentication
/api/auth/register	POST	User registration
/api/health	GET	System health check

WebSocket — Real-time push

Single WebSocket endpoint at /ws. Authenticated with JWT on connect. Server pushes updates whenever Redis Streams have new messages. No polling from the client side.

Message Type	Payload
regime_update	Regime label, confidence, transition probabilities, time-in-regime
microstructure_update	OFI, VPIN, Kyle Lambda, spread components
altdata_signal	Source, value, z-score, confluence contribution
llm_insight	Causal summary, affected assets, direction, confidence
alert	Alert type, message, timestamp, triggering signal

9. Per-User Dashboard + Dynamic Symbols

Every user gets an isolated dashboard at `vektorlabs.xyz/dashboard/{username}`. Their configuration is fully personal. Their symbol watchlist is their own. No user's configuration affects any other user.

Dynamic Symbol Subscriptions

Users add any valid Binance futures symbol up to their plan limit. The backend manages WebSocket streams dynamically based on active subscriber counts.

Plan	Symbol Limit	Price
Trial	3 symbols	Free
Standard	10 symbols	\$47/mo
Unlimited	No limit	\$97/mo

How dynamic subscriptions work

- User adds a symbol via POST `/api/users/{username}/symbols`.
- Backend validates symbol against Binance exchangeInfo cache (refreshed hourly).
- Backend checks `active_symbols` table. If `subscriber_count` is 0, a new WebSocket stream task is started dynamically. No restart required.
- `subscriber_count` incremented. Stream is now shared across all users tracking that symbol.
- When user removes a symbol, `subscriber_count` decrements. If it hits 0, the stream closes and the symbol is removed from `active_symbols`.

User Settings Page

Accessible at `vektorlabs.xyz/dashboard/{username}/settings`. Users control their own configuration. System-level parameters are completely hidden and inaccessible.

Setting	User Controllable	Description
Watched symbols	Yes	Add/remove up to plan limit
Discord webhook	Yes	Personal webhook for alerts
Alert thresholds	Yes	Per-signal notification triggers
Dashboard layout	Yes	Widget arrangement and visibility
Reddit subreddits	Yes	Which subreddits to monitor
Google Trends keywords	Yes	Custom search terms
FRED macro series	Yes	Which macro indicators to display
Display timezone	Yes	Local timezone for timestamps
HMM model parameters	No	Admin only — protected

Regime thresholds	No	Admin only — protected
Feature windows	No	Admin only — protected

10. Storage Design

Two storage systems. TimescaleDB for all time-series data with automatic partitioning and compression. PostgreSQL (same instance) for relational data — users, settings, subscriptions. Redis for real-time state only.

TimescaleDB — Hypertables

Table	Type	Partition Interval	Retention
microstructure_raw	Hypertable	1 day	30 days
regime_states	Hypertable	1 day	Forever
alt_data_signals	Hypertable	1 day	90 days
llm_insights	Regular	—	Forever

All hypertables have a composite index on (symbol, timestamp DESC). Compression policy activates on chunks older than 7 days. Raw tick data compresses approximately 90% with columnar compression.

PostgreSQL — Relational Tables

Table	Purpose
users	Authentication: id, username, email, password hash, plan, created_at
user_settings	Per-user config: user_id, symbols[], discord_webhook, alert_config (JSONB), display_timezone, reddit_subreddits[], fred_series[], trends_keywords[]
active_symbols	Stream registry: symbol, subscriber_count, stream_active, last_tick
subscriptions	Billing: user_id, plan, status, oxapay_id, trial_ends, next_billing
system_config	Behavioural config: key, value, updated_at — no YAML, all runtime

Redis — Real-time only

Key Pattern	Type	Contents	TTL / maxlen
microstructure:{symbol}	Stream	OFI, VPIN, spread metrics	maxlen 10,000
regime:{symbol}	Stream	Regime state + probabilities	maxlen 5,000
altdata:signals	Stream	Alt data signal points	maxlen 1,000
llm:insights	Stream	Causal insight objects	maxlen 500
latest:{symbol}	String	Latest microstructure state	TTL 60s
valid_symbols	Set	Binance futures symbol list	TTL 1 hour
active_symbols	Hash	symbol -> subscriber count	No TTL

11. Configuration — No YAML

All configuration is runtime. No YAML files. No hardcoded values in application code. Three tiers of configuration, each stored differently.

Tier	What it contains	Where stored	Who can change it
Secrets	API keys, DB DSN, Redis URL, JWT secret	.env file only	Developer
Infrastructure	Connection pool sizes, retry intervals, ports	.env file	Developer
Behavioural	Feature windows, HMM params, polling intervals	system_config table	Admin API
User	Symbols, alerts, layout, subreddits, keywords	user_settings table	User dashboard

core/config.py — The single source of truth

Every other module imports configuration exclusively from core/config.py. No module reads from .env or the database directly. config.py loads from .env at startup, then fetches behavioural config from system_config table into memory. A reload() method refreshes behavioural config from the database without restarting the process.

Rule: if you find yourself importing os.environ or reading a YAML file anywhere outside core/config.py, it is wrong. Move it.

12. Infrastructure + Deployment

Single Hetzner VPS. Docker Compose manages all services. Nginx handles TLS termination and reverse proxying. Systemd manages the Docker Compose process on the host.

Docker Compose Services

Service	Image	Purpose
vektor-core	Custom Python 3.11	Main async engine — all modules + API
timescaledb	timescale/timescaledb-ha:pg16	Time-series + relational storage
redis	redis:7-alpine	Real-time signal bus + cache
nginx	nginx:alpine	TLS termination + reverse proxy
frontend	node:20-alpine (build)	React build served via nginx

Startup Sequence

Order matters. A failure at any step before the extractors start should crash the process loudly rather than starting with broken configuration.

- Step 1 — Load environment variables from `.env`.
- Step 2 — Establish TimescaleDB connection pool (asyncpg).
- Step 3 — Establish Redis connection.
- Step 4 — Load behavioural config from `system_config` table into memory.
- Step 5 — Run pending database migrations.
- Step 6 — Fetch valid Binance symbols into Redis cache.
- Step 7 — Start WebSocket extractors for all symbols in `active_symbols`.
- Step 8 — Start all intelligence module consumers.
- Step 9 — Start FastAPI server.

13. Data Sources Reference

Source	Data	Cost	URL
Binance WS	L2 order book, trades, streams	Free	fapi.binance.com
Binance REST	Funding rate, OI, liquidations	Free	fapi.binance.com/fapi/v1
FRED API	CPI, yield curve, unemployment	Free	api.stlouisfed.org
Reddit PRAW	Subreddit posts, scores, comments	Free	praw.readthedocs.io
Google Trends	Search volume via pytrends	Free	github.com/GeneralMills/pytrends
Blockchain.com	On-chain volume, exchange flows	Free	api.blockchain.info
Mempool.space	BTC fees, mempool, UTXO data	Free	mempool.space/api
SEC EDGAR	8-K, 10-Q filings, transcripts	Free	efts.sec.gov
NewsAPI	News headlines and article bodies	Free*	newsapi.org
OpenRouter	LLM inference (Mistral, LLaMA)	Free*	openrouter.ai

** Free tier with rate limits. Sufficient for a single-instance deployment.*

14. Build Order + Study Map

Build sequentially. Each phase produces something functional before the next phase starts. Do not build in parallel.

Build Phases

Phase	What to build	Gate to next phase
0	core/config.py + Docker Compose TimescaleDB + Redis running locally Migrations 001 and 002	DB reachable, config loads cleanly
1	ingestion/ ETL pipeline Binance WS streaming to Redis Raw data persisting to TimescaleDB	Ticks flowing, DB writing, no crashes in 1 hour
2	modules/microstructure/ OFI, VPIN, Kyle Lambda live	Metrics publishing to Redis Stream
3	modules/regime/ HMM trained on historical data Real-time regime classification	Regime state publishing with confidence > 0
4	modules/altdata/ All ingestors + extractors Confluence engine	Confluence score publishing to Redis
5	modules/llm_reasoner/ SEC + news fetchers Causal extraction prompts	Insights publishing to Redis
6	api/ + frontend/ REST endpoints + WebSocket React dashboard	Dashboard live at localhost:3000
7	Auth + per-user dashboards Dynamic symbol subscriptions OxaPay billing integration	One real user can subscribe and pay

Study Map — Minimum required, in order

Topic	Resource	Needed for
VPIN	Easley et al. 2012 — SSRN 1695596	Phase 2
Glosten-Milgrom spread	Glosten-Milgrom 1985 — JSTOR	Phase 2
Kyle Lambda	Kyle 1985 — JSTOR	Phase 2
HMM tutorial	Rabiner 1989 — CMU PDF	Phase 3
hmmlearn	hmmlearn.readthedocs.io	Phase 3
Granger causality	Granger 1969 — MIMUW PDF	Phase 4
Prompt engineering	docs.anthropic.com/en/docs/...	Phase 5
TimescaleDB hypertables	docs.timescale.com	Phase 0
Redis Streams	redis.io/docs/latest/...	Phase 0
asyncpg pool	magicstack.github.io/asyncpg	Phase 0

Rule: study just enough to start the phase, build until you hit a wall, then study exactly what the wall requires. Do not front-load theory.